

5.1)

A component-based architecture views the system as a collection of loosely coupled components that provide services to one another. A service-oriented architecture is similar, except that the pieces are implemented as services, often running on separate computers that communicate across a network. The two are similar, but the pieces are more separated in a service-oriented architecture.

5.2)

This is a simple, self-contained application, so no remote services or database are required. That means client-server, multitier, component-based, and service-oriented architectures are probably overkill, so this application would be easiest to build as a simple monolithic rule-based (data-centric) application.

5.4)

This scenario is more complicated than the previous one, but the user interface remains largely unchanged. The key differences are: (1) The program needs to exchange information with another instance of the program across the Internet, and (2) There's no computer opponent.

Removing the computer opponent means the program doesn't need distributed pieces.

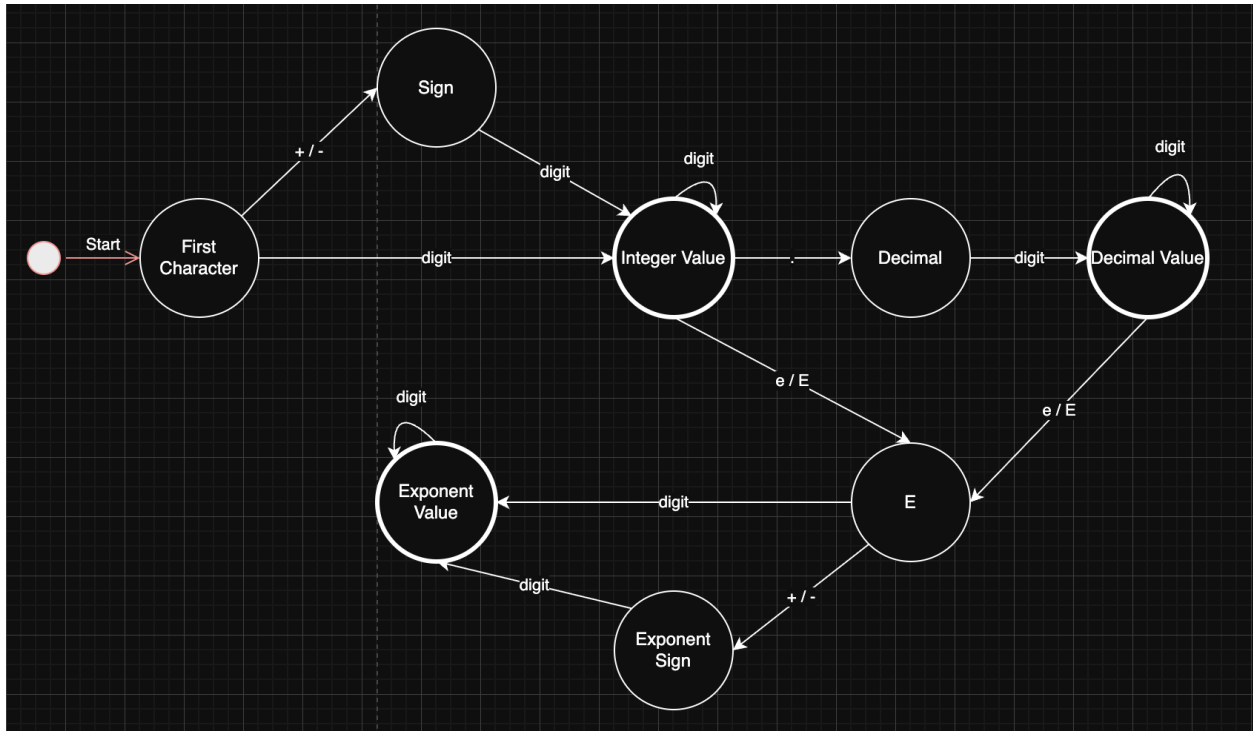
You could use web services to allow two programs to communicate over the Internet. That would make the application a monolithic, rule-based (data-centric) service-oriented application.

5.6)

The ClassyDraw application can store each drawing in a separate file, so it doesn't need much of a database. Operating system tools can let the user manage files. For example, they let the user delete old files and make backup copies of files.

The program could create a temporary file while the user is editing a drawing. Then, if the program crashes or is ended prematurely, it could ask the user if it should restore the temporary file the next time it starts.

5.8)



6.1)

The classes all represent things that are drawn, so they share properties needed for drawing.

- Those include foreground color and background color. All the classes can also define their drawing position by storing the upper-left corner, a width, and a height.
- Some classes need extra data to draw their particular type of shape, and the classes won't share that data. For example, the Text class needs font information and the string to draw. The Star class needs to know how many points to give the star.
- Some properties can be shared by some classes and not others. Rectangle, Ellipse, and Star can be filled, so they need a fill color. The classes that draw lines (Line, Rectangle, Ellipse, and Star) also need line properties (such as line thickness and dash style).
- Shared properties should be implemented in a parent class, while non-shared properties should be implemented in the child classes

6.2)

